NEUROBYTE LAB

PYTHON

Basics to Advanced: A Practical Learning Guide

Concepts • Examples • Best Practices • Practice Tasks

A beginner-friendly reference for students, self-learners, and aspiring developers.

Code • Learn • Innovate • Grow

Prepared for learning and revision. Examples target modern Python 3.

How to Use This Guide

This guide moves from core syntax to advanced language features. Read one section, type every example yourself, change the inputs, and explain the output in your own words. Do not only copy and paste.

• **Beginner:** Chapters 1–8 build the foundations.

• **Intermediate:** Chapters 9–15 cover reusable code, files, errors, modules, and object-oriented programming.

• **Advanced:** Chapters 16–22 cover iterators, generators, decorators, context managers, typing, testing, concurrency, and performance.

• **Project practice:** Chapter 23 gives mini-project ideas and a revision checklist.

Table of contents

1. Getting started and syntax
2. Variables and data types
3. Operators and expressions
4. Strings
5. Collections: list, tuple, set, dictionary
6. Conditions and loops
7. Functions and scope
8. Comprehensions and unpacking
9. Modules, packages, and pip
10. Files and paths
11. Exceptions and debugging
12. Object-oriented programming
13. Dataclasses, enums, and properties
14. Working with JSON and CSV
15. Regular expressions and dates
16. Iterables, iterators, and generators
17. Decorators and closures
18. Context managers
19. Type hints and modern Python
20. Testing and code quality
21. Concurrency and parallelism
22. Performance and memory
23. Mini-projects and practice roadmap
24. Quick reference

1. Getting Started and Syntax

Python is a general-purpose programming language known for readable syntax. Install Python 3 from python.org, then run a .py file with `python filename.py` or `py filename.py` on many Windows systems.

Your first program

```
print("Hello, world!")
```

Comments and indentation

```
# This is a one-line comment
name = "Asha"

if name:
    print("Hello", name)  # Indentation defines the block
```

Python uses indentation rather than braces to group statements. Use four spaces per indentation level. Keep naming consistent and avoid mixing tabs and spaces.

Useful built-ins

```
print("Hello")
len("Python")       # 6
type(42)            # <class 'int'>
help(str)           # documentation for str
dir(list)           # names available on list
```

Remember: Python is case-sensitive: `score`, `Score`, and `SCORE` are different names.

2. Variables and Data Types

A variable is a name bound to an object. Python is dynamically typed, so a name can later refer to an object of another type.

```
student = "Maya"
age = 20
height = 1.65
is_student = True
nothing = None

print(type(student))
```

Core types

• **int:** whole numbers, e.g. `count = 12`

• **float:** decimal numbers, e.g. `price = 19.5`

• **str:** text, e.g. `city = 'Chittoor'`

• **bool:** `True` or `False`

• **NoneType:** absence of a value, represented by `None`

Conversion and input

```
age_text = input("Enter age: ")  # input returns a string
age = int(age_text)
print(age + 1)

number = float("3.14")
text = str(100)
```

Naming conventions

Use descriptive `snake_case` names for variables and functions, `PascalCase` for classes, and uppercase names for constants by convention. Avoid names such as `list`, `str`, or `sum` for your own variables because they shadow built-ins.

Remember: Use `==` to compare values; `=` assigns a value.

3. Operators and Expressions

Arithmetic

```
a, b = 17, 5
print(a + b)   # 22
print(a - b)   # 12
print(a * b)   # 85
print(a / b)   # 3.4
print(a // b)  # 3: floor division
print(a % b)   # 2: remainder
print(a ** b)  # 1419857
```

Comparisons and logic

```
x = 8
print(x > 5 and x < 10)  # True
print(x == 8)            # True
print(not False)         # True
```

Membership and identity

```
"py" in "python"       # True
3 in [1, 2, 3]         # True

a = [1, 2]
b = a
c = [1, 2]
a is b                 # True: same object
a == c                 # True: equal values
a is c                 # False: different objects
```

Remember: Use `is None` to check for None. Use `==` for normal value equality.

4. Strings

Creating and indexing strings

```
language = "Python"
print(language[0])      # P
print(language[-1])     # n
print(language[1:4])    # yth
print(language[::-1])   # nohtyP
```

Common methods

```
message = "  learn python  "
message.strip()             # removes outside whitespace
message.upper()             # uppercase
message.replace("python", "SQL")
"one,two,three".split(",")
"-".join(["2026", "10", "24"])
```

f-strings

```
name = "Maya"
score = 92.456
print(f"{name} scored {score:.1f}%")
# Maya scored 92.5%
```

Strings are immutable: a string method returns a new string rather than changing the original. Use `repr(value)` when debugging whitespace or special characters.

5. Collections: List, Tuple, Set, Dictionary

Lists: ordered and mutable

```
numbers = [10, 20, 30]
numbers.append(40)
numbers.extend([50, 60])
numbers.insert(0, 5)
numbers.remove(20)
last = numbers.pop()
numbers.sort()
print(numbers)
```

Tuples: ordered and immutable

```
point = (4, 7)
x, y = point
single_item = (5,)  # comma makes it a tuple
```

Sets: unique elements

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)  # union
print(a & b)  # intersection
print(a - b)  # difference
```

Dictionaries: key-value mapping

```
student = {"name": "Maya", "score": 90}
student["score"] = 95
student["city"] = "Puttur"
print(student.get("email", "Not supplied"))

for key, value in student.items():
    print(key, value)
```

Choosing a collection

• Use a list for an ordered sequence that may change.

• Use a tuple for a fixed record or immutable sequence.

• Use a set for uniqueness and set operations.

• Use a dictionary to look up values by key.

Remember: A shallow copy duplicates the outer container, not nested objects. Use `copy.deepcopy()` only when a deep copy is actually needed.

6. Conditions and Loops

```
temperature = 28
if temperature >= 35:
    print("Hot")
elif temperature >= 25:
    print("Warm")
else:
    print("Cool")
```

for, while, break, continue

```
for number in range(1, 6):  # 1 through 5
    if number == 3:
        continue
    print(number)

count = 3
while count > 0:
    print(count)
    count -= 1
```

Looping over collections

```
names = ["Asha", "Maya", "Ravi"]
for index, name in enumerate(names, start=1):
    print(index, name)

prices = {"pen": 10, "book": 80}
for item, price in prices.items():
    print(item, price)
```

Use `break` to exit a loop, `continue` to skip to the next iteration, and the loop `else` clause when a loop finishes without hitting `break`.

7. Functions and Scope

```
def greet(name, greeting="Hello"):
    """Return a friendly greeting."""
    return f"{greeting}, {name}!"

print(greet("Maya"))
print(greet("Ravi", greeting="Welcome"))
```

Arguments

```
def report(name, *scores, city="Unknown", **details):
    print(name, scores, city, details)

report("Maya", 80, 90, city="Puttur", active=True)
```

`*scores` gathers extra positional arguments into a tuple. `**details` gathers extra keyword arguments into a dictionary. Keyword-only parameters appear after `*` or `*args`.

Avoid mutable default arguments

```
# Avoid: def add_item(item, items=[]):
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Scope and lambda

```
rate = 2

def multiply(value):
    factor = 3
    return value * factor

square = lambda x: x * x
print(square(5))
```

Local variables belong to a function call. Global names live at module scope. Prefer passing values and returning results rather than modifying globals.

8. Comprehensions and Unpacking

```
squares = [n * n for n in range(6)]
evens = [n for n in range(20) if n % 2 == 0]
unique_lengths = {len(word) for word in ["cat", "tree", "dog"]}
lookup = {n: n * n for n in range(5)}

first, second, *remaining = [10, 20, 30, 40]
left, right = right, left  # swap names
```

Generator expressions

```
total = sum(n * n for n in range(1_000_000))
```

Comprehensions are concise, but keep them readable. If a comprehension becomes deeply nested or hard to understand, use a normal loop.

9. Modules, Packages, and pip

```
# Import a module
import math
print(math.sqrt(81))

# Import a specific name
from pathlib import Path
print(Path(".").resolve())
```

Create your own module

If `helpers.py` contains `def clean_text(text): ...`, another file in the same importable location can use `from helpers import clean_text`. Keep executable scripts protected with the main guard:

```
def main():
    print("Program starts here")

if __name__ == "__main__":
    main()
```

Virtual environments and packages

```
# Create a virtual environment
python -m venv .venv

# Windows PowerShell activation
.venv\Scripts\Activate.ps1

# macOS/Linux activation
source .venv/bin/activate

python -m pip install requests
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

Use a virtual environment per project to isolate dependencies. Install packages from trusted sources and check project documentation for supported Python versions.

10. Files and Paths

Read and write text safely

```
from pathlib import Path

path = Path("notes.txt")
path.write_text("Python is useful.\n", encoding="utf-8")
content = path.read_text(encoding="utf-8")
print(content)
```

Using with and file modes

```
with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.rstrip())

with open("log.txt", "a", encoding="utf-8") as file:
    file.write("Program finished\n")
```

Common modes: `r` read, `w` overwrite/create, `a` append, and `x` create only if the file does not already exist. Use `with` so the file is closed reliably.

Paths and folders

```
from pathlib import Path

folder = Path("data")
folder.mkdir(exist_ok=True)
for file_path in folder.glob("*.csv"):
    print(file_path.name)
```

11. Exceptions and Debugging

```
try:
    value = int(input("Enter a whole number: "))
    result = 100 / value
except ValueError:
    print("Please enter a valid integer.")
except ZeroDivisionError:
    print("The value must not be zero.")
else:
    print("Result:", result)
finally:
    print("Finished.")
```

Raise meaningful errors

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Insufficient balance")
    return balance - amount
```

Catch specific exceptions instead of using a bare `except`. Do not silently ignore failures. Use tracebacks, a debugger, small test inputs, and logging to locate bugs.

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Application started")
logging.warning("Unexpected input received")
```

12. Object-Oriented Programming

A class defines a type; an object is an instance of that class. Use OOP when it helps represent data and behavior together.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self.balance += amount

    def summary(self):
        return f"{self.owner}: {self.balance}"

account = BankAccount("Maya", 100)
account.deposit(50)
print(account.summary())
```

Inheritance and overriding

```
class SavingsAccount(BankAccount):
    def __init__(self, owner, balance=0, interest_rate=0.02):
        super().__init__(owner, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        self.balance += self.balance * self.interest_rate
```

• **Encapsulation:** expose a clear interface and keep internal details manageable.

• **Inheritance:** derive a specialized class from a base class when the relationship is genuine.

• **Polymorphism:** different objects can respond to the same method name.

• **Composition:** build a class using other objects; often simpler than deep inheritance.

Remember: Python uses conventions such as `_internal_name` to indicate non-public implementation details; this is not strict access control.

13. Dataclasses, Enums, and Properties

```
from dataclasses import dataclass

@dataclass
class Transaction:
    transaction_id: str
    amount: float
    category: str = "general"

tx = Transaction("TX100", 250.0)
print(tx)
```

Properties

```
class Product:
    def __init__(self, price):
        self.price = price

    @property
    def price(self):
        return self._price

    @price.setter
    def price(self, value):
        if value < 0:
            raise ValueError("Price cannot be negative")
        self._price = value
```

Enums

```
from enum import Enum

class RiskLevel(Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"

risk = RiskLevel.HIGH
```

Dataclasses reduce boilerplate for data-focused classes. Properties allow attribute-like access while validating or calculating values. Enums represent a fixed set of named choices.

14. Working with JSON and CSV

```
import json

record = {"id": "TX1", "amount": 125.5, "flagged": False}
text = json.dumps(record, indent=2)
restored = json.loads(text)
print(restored["amount"])
```

```
import csv

with open("transactions.csv", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["transaction_id"], row["amount"])
```

JSON represents structured data using objects, arrays, strings, numbers, booleans, and null. CSV is useful for tabular data. Validate external data before trusting it, and handle missing or malformed fields.

15. Regular Expressions and Dates

```
import re

text = "Contact: learner@example.com"
match = re.search(r"[\w.-]+@[\w.-]+\.\w+", text)
if match:
    print(match.group())
```

Use regular expressions for pattern matching, not as a replacement for a full parser when the format is complex. Common tools include `re.search`, `re.findall`, `re.sub`, and compiled patterns.

```
from datetime import date, datetime, timedelta

today = date.today()
tomorrow = today + timedelta(days=1)
stamp = datetime.now().isoformat(timespec="seconds")
print(today, tomorrow, stamp)
```

For timestamps shared across systems, prefer timezone-aware datetimes and use UTC where appropriate. Display formatting and parsing are available through `strftime` and `strptime`.

16. Iterables, Iterators, and Generators

An iterable can provide an iterator. An iterator yields values one at a time and remembers its progress. A generator function uses `yield` to produce values lazily.

```
numbers = [10, 20, 30]
iterator = iter(numbers)
print(next(iterator))
print(next(iterator))
```

```
def count_up_to(limit):
    current = 1
    while current <= limit:
        yield current
        current += 1

for value in count_up_to(3):
    print(value)
```

Why generators?

• They can process large sequences without building the entire result in memory.

• They are useful for streams, pipelines, and reading data incrementally.

• A generator is usually consumed once; create a new one when you need a fresh pass.

```
squares = (n * n for n in range(1_000_000))
first_five = [next(squares) for _ in range(5)]
```

Remember: A generator expression uses parentheses. A list comprehension uses square brackets and builds a list immediately.

17. Decorators and Closures

A closure is a function that remembers values from its enclosing scope. A decorator wraps a function to add behavior without changing the original function's main logic.

```
def make_multiplier(factor):
    def multiply(value):
        return value * factor
    return multiply

double = make_multiplier(2)
print(double(8))
```

```
from functools import wraps
import time

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed = time.perf_counter() - start
            print(f"{func.__name__}: {elapsed:.6f}s")
    return wrapper

@timed
def calculate():
    return sum(range(10000))

calculate()
```

Decorators are commonly used for logging, timing, caching, authorization, and framework registration. `functools.wraps` preserves useful metadata from the wrapped function.

18. Context Managers

A context manager handles setup and cleanup around a block, such as closing a file or releasing a lock. The `with` statement is the usual interface.

```
from contextlib import contextmanager

@contextmanager
def managed_message():
    print("Start")
    try:
        yield "inside the block"
    finally:
        print("Cleanup")

with managed_message() as message:
    print(message)
```

The `finally` section runs even when the block raises an exception. Use context managers for resources that must be released reliably.

19. Type Hints and Modern Python

```
def average(values: list[float]) -> float:
    if not values:
        raise ValueError("values cannot be empty")
    return sum(values) / len(values)

def find_user(user_id: int) -> str | None:
    if user_id == 1:
        return "Maya"
    return None
```

Type hints improve readability and enable static analysis tools, but Python does not automatically enforce most annotations at runtime. For reusable APIs, consider `TypedDict`, `Protocol`, `Literal`, and generics when they make contracts clearer.

```
from typing import TypedDict

class UserRecord(TypedDict):
    username: str
    active: bool

user: UserRecord = {"username": "maya", "active": True}
```

Useful modern language features

• F-strings for readable formatting.

• Structural pattern matching with `match`/`case` in Python 3.10+.

• Union types such as `int | None` in Python 3.10+.

• Dataclasses for data-oriented classes.

• Pathlib for readable filesystem paths.

```
status = 200
match status:
    case 200:
        print("OK")
    case 404:
        print("Not found")
    case _:
        print("Other status")
```

20. Testing and Code Quality

Unit tests with unittest

```
import unittest

def add(a, b):
    return a + b

class TestAdd(unittest.TestCase):
    def test_two_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

if __name__ == "__main__":
    unittest.main()
```

Test normal cases, edge cases, invalid input, and failures. A test should be repeatable and should not depend on unrelated tests. For larger projects, pytest is a popular third-party testing framework.

Quality habits

• Use descriptive names and small functions with one clear responsibility.

• Follow PEP 8 style conventions.

• Format code consistently and run a linter where appropriate.

• Write docstrings for public functions and classes.

• Keep secrets such as API keys out of source control.

• Use Git commits to record small, meaningful changes.

```
# Run a unittest file
python -m unittest

# Compile-check a Python file for syntax errors
python -m py_compile app.py
```

21. Concurrency and Parallelism

Concurrency helps organize tasks that overlap. Parallelism means tasks execute at the same time. The best approach depends on whether the work is I/O-bound or CPU-bound.

Threads for I/O-bound work

```
from concurrent.futures import ThreadPoolExecutor
import time

def fetch_simulated(item):
    time.sleep(0.2)  # stands in for network or disk waiting
    return f"done: {item}"

with ThreadPoolExecutor(max_workers=4) as pool:
    results = list(pool.map(fetch_simulated, range(8)))
print(results)
```

Processes for CPU-bound work

```
from concurrent.futures import ProcessPoolExecutor

def square(n):
    return n * n

if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:
        print(list(pool.map(square, range(10))))
```

Asyncio for many asynchronous I/O tasks

```
import asyncio

async def task(name):
    await asyncio.sleep(0.2)
    return f"finished {name}"

async def main():
    results = await asyncio.gather(task("A"), task("B"))
    print(results)

if __name__ == "__main__":
    asyncio.run(main())
```

On standard CPython builds, the Global Interpreter Lock can limit parallel execution of Python bytecode in threads for CPU-heavy code. Threads can still be excellent for network and file I/O. Processes add overhead and require care with pickling and platform behavior. Async code requires awaitable APIs.

22. Performance and Memory

• Measure before optimizing. Use `time.perf_counter()` for elapsed time.

• Use `timeit` for small repeatable timing experiments.

• Use `cProfile` to find functions consuming time.

• Prefer suitable data structures and algorithms over micro-optimizations.

• Generators can reduce memory use for large streams.

• Avoid repeated expensive work; caching may help when inputs repeat.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(30))
```

Caching is appropriate only when repeated calls with the same inputs can safely reuse results. Be cautious with mutable results, external state, and unbounded caches.

Big-O basics

• O(1): constant-time operation, e.g. average dictionary lookup.

• O(log n): logarithmic growth, e.g. binary search on sorted data.

• O(n): linear scan through n items.

• O(n log n): common efficient comparison sorting complexity.

• O(n^2): nested comparisons may become expensive as input grows.

23. Mini-Projects and Practice Roadmap

Beginner projects

• Calculator: functions, input validation, conditions.

• Number guessing game: loops, random numbers, comparisons.

• To-do list in the terminal: lists, functions, file storage.

Intermediate projects

• Expense tracker: CSV or JSON storage, dates, summaries, exceptions.

• Student marks manager: dictionaries, sorting, reusable functions, tests.

• Log analyzer: file reading, string methods, counters, regular expressions.

Advanced projects

• API client: HTTP library, environment variables, retries, structured errors, tests.

• Concurrent file processor: pathlib, executors, logging, performance measurements.

• Transaction anomaly analysis prototype: pandas/scikit-learn, preprocessing, metrics, clear separation between data preparation, training, prediction, and explanation.

Suggested 6-week learning plan

Week	Focus	Deliverable
1	Syntax, variables, strings, collections	Small quiz + calculator
2	Conditions, loops, functions, comprehensions	Number game
3	Modules, files, errors, JSON/CSV	Expense tracker
4	OOP, dataclasses, typing, testing	Student marks manager + tests
5	Iterators, generators, decorators, context managers	Log processing tool
6	Concurrency, performance, project polish	Documented GitHub project

Practice tasks

• Write a function that counts vowels in a string.

• Remove duplicates from a list while preserving order.

• Count words in a text file and report the five most frequent.

• Create a class that validates a transaction amount.

• Write unit tests for a function that classifies risk as low, medium, or high.

• Use a generator to read a large file line by line.

• Compare two approaches with `timeit` and explain the result.

24. Quick Reference

Common built-ins

`print`, `input`, `len`, `range`, `enumerate`, `zip`, `sorted`, `sum`, `min`, `max`, `any`, `all`, `map`, `filter`, `isinstance`, `type`, `open`, `repr`, `round`.

Common collection methods

List: `append`, `extend`, `insert`, `remove`, `pop`, `sort`, `reverse`.
String: `strip`, `split`, `join`, `replace`, `find`, `startswith`, `endswith`, `lower`, `upper`.
Dictionary: `get`, `keys`, `values`, `items`, `update`, `pop`.
Set: `add`, `remove`, `discard`, `union`, `intersection`.

Common commands

```
python --version
python script.py
python -m venv .venv
python -m pip install package_name
python -m pip freeze > requirements.txt
python -m unittest
python -m py_compile script.py
```

Reliable learning resources

- Official Python Tutorial
- Python Standard Library Reference
- PEP 8 Style Guide
- Python Packaging User Guide

This guide is a learning reference, not a replacement for official documentation. Python and third-party packages evolve, so check the documentation for version-specific details.

NeuroByte Lab — Code • Learn • Innovate • Grow